

ADR-0007 — Presentational/container split with a hard cap on event-forwarding depth

ACCEPTED Date: 2026-09-21 · Deciders: Nicolà Widmer

Context and Problem Statement

The Angular SPA (`frontend/src/app/`) is organised as feature folders (`features/songs`, `features/playlists`, `features/player`) plus a `shared/` slice for components used across features (`shared/songs/*`). As the song-row and player controls grew their own sub-components (a menu, a drag handle, a scrubber, a turntable), user intent — “play this song”, “delete this song”, “seek to here” — has to travel from a small presentational leaf up to whichever component owns the mutation or the playback store. Angular’s idiomatic tool for that is `output()`, re-emitted one level at a time. Left unchecked, that re-emission chain grows with the component tree: every new wrapper adds another `(event)="event.emit($event)"` line, in every component the intent passes through, for every kind of intent that component’s children can raise. A `play / edit / delete` intent forwarded through five layers means five places to keep in sync when a sixth is added, and a bug in hop three silently swallows the event with no compiler error.

Decision Drivers

- One developer: every re-emitting hop is a place a rename or a new event kind can be forgotten, with no compiler error when it is.
- Angular’s own idiom (dumb components + outputs) is worth keeping for real reuse and testability — the answer is a **depth limit**, not “inject a service everywhere”.
- Some state is genuinely global to the shell, not owned by any one route (playback — the bottom bar and the drawer both render it).
- Some state is genuinely owned by exactly one container per screen (which song is mid-edit, which mutation is in flight) and has no reason to live anywhere else.

Considered Options

1. **Presentational/container split, output events capped at two forwarding hops.** Leaf components (`SongMenuComponent`, `PlayerTransportComponent`, ...) only emit. A layout component in between (`SongRowComponent`, `SongListComponent`) may re-emit the **same** event unchanged, once, when it exists purely to lay out repeated children — never to add a hop for its own sake. The chain always terminates at one **smart** component (a “panel” or a route component) that owns the query/mutation or calls a shared service; nothing re-emits past that point. Cross-cutting client state that many, unrelated parts of the tree need (playback) lives in an injectable signal-based service instead of being threaded through inputs/outputs at all.
2. **Thread everything through inputs/outputs regardless of depth** — the default if no one pays attention to it; rejected as the status quo this ADR is reacting to.
3. **One global store for everything** (NgRx/Akita-style), components dispatch actions instead of emitting outputs — rejected as disproportionate for a two-route app; it trades re-emission chains for indirection through action types, and most state here (a query result, a dialog’s open song) is cleanly owned by one component already.
4. **inject() the owning container’s mutation logic directly from every leaf** — rejected: it turns `SongMenuComponent` into something that can only be used inside `SongListPanelComponent`, destroying the presentational/reusable split option 1 keeps.

Decision Outcome

Chosen: **option 1**.

- Presentational leaves stay presentational: `SongMenuComponent`, `SongRowComponent`, `PlayerTransportComponent`, `PlaylistCardComponent`, etc. take `input()`s and raise `output()`s; none of them injects a query, a mutation, or a route param.
- Exactly one **smart** component per concern owns the TanStack Query query/mutations and is where an event chain ends: `SongListPanelComponent` (`play / edit / delete / add-to-playlist / remove-from-playlist / reorder`, shared by the library route and every playlist-detail route), `PlaylistsPageComponent` (`rename / delete`),

`SongEditDialogComponent` and `PlaylistCreateDialogComponent` (their own form + mutation, opened/closed via a single input/output pair).

- A layout-only component in between (`SongListComponent` laying out `SongRowComponents` in a `p-dataview` or a `cdkDropList`) may re-emit an event unchanged — that is **one** hop, still short, and exists only because the same row markup is reused in two layouts (drag-reorderable and plain). `SongMenuComponent` → `SongRowComponent` → `SongListComponent` → `SongListPanelComponent` is the deepest chain in the app: three forwards, each one an unchanged re-emit, terminating at the one component that knows what “delete” means. Nothing here forwards an event because that used to be one hop shorter and nobody revisited it.
- Playback is the one piece of state several unrelated parts of the shell render at once (the sticky bottom bar and the expanded drawer, simultaneously mounted). It lives in `PlayerService` (`providedIn: 'root'`, §8) — a signal store wrapping one `<audio>` element — injected directly by both. No `@Input/@Output` chain could reach both consumers without a common ancestor re-emitting to both, which is exactly the pattern this ADR caps.
- Rule of thumb enforced in review, not by tooling: if adding a feature would make an event’s forwarding chain exceed **two** re-emitting hops before it reaches a smart component, that is the signal to either promote the interaction to a shared service (if truly cross-cutting, like playback) or restructure so the smart component sits closer to where the event originates — not to add a third hop.

Consequences

- Good: every event’s owner is found by reading at most three components (source, at most one relay, the smart component) — no chain-of-custody spelunking through the tree to find where a click ends up.
- Good: presentational components stay trivially reusable and unit-testable — `SongMenuComponent` has no idea whether it is deleting a library song or removing a playlist entry; that is `SongListPanelComponent`’s `variant` input to decide.
- Good: cross-cutting client state (playback) has one clear home instead of being routed through whichever component happens to be the nearest common ancestor today.
- Bad: the two-hop rule is a review convention, not a lint rule — nothing stops a chain from silently growing to three hops as new wrappers are added; revisit if that turns out to happen often (§11).
- Bad: reaching for `PlayerService` for something that **isn’t** truly cross-cutting would reintroduce the “just inject it” problem option 1 rejects — the line is “rendered by more than one simultaneously-mounted component”, not “would save a hop”.

More Information

Related: ADR-0006 (the other half of state — server state lives in TanStack Query, this ADR is about client state and event flow). See §5 for the frontend’s building blocks and §6 for a traced example (play a song from the library).